

Assignment 4: Hashing

NAME: YUE CHENGHAO

Matric No: 227154

Date: 12,JAN,2026

1. Task 1: Illustrating Hashing with Open Addressing.....	1
1.1. Linear Probing.....	1
1.2. Double Hashing.....	3
1.3. Comparison.....	4
2. Task 2: Hash Table Implementation and Search.....	4
2.1. Java Implementation.....	5
2.2. Example Simulation.....	7
3. Conclusion.....	8

1. Task 1: Illustrating Hashing with Open Addressing

Problem Statement: Insert the keys 10, 22, 31, 4, 15, 28, 17, 88, 59 into a hash table of length $m=11$ using open addressing. The primary hash function is $h_1(k) = k \bmod m$. Illustrate the results using both linear probing and double hashing, where the secondary hash function for double hashing is $h_2(k) = 1 + (k \bmod (m-1))$.

1.1. Linear Probing

In linear probing, when a collision occurs at an index $h(k)$, the algorithm probes the next available slot sequentially: $(h(k) + i) \bmod m$, where i is the probe number (0, 1, 2, ...). The process continues until an empty slot is found.

Given $m = 11$ and $h_1(k) = k \bmod 11$:

Key	$h1(k) = k \bmod 11$	Probe Sequence (pos = $(h1(k) + i) \bmod 11$)	Final Index	Hash Table State (after insertion)
10	10	$(10 + 0) \bmod 11 = 10$	10	[, , , , , , , , , 10]
22	0	$(0 + 0) \bmod 11 = 0$	0	[22, , , , , , , , 10]
31	9	$(9 + 0) \bmod 11 = 9$	9	[22, , , , , , , 31, 10]
4	4	$(4 + 0) \bmod 11 = 4$	4	[22, , , , 4, , , , 31, 10]
15	4	$(4 + 0) \bmod 11 = 4$ (collision with 4)		
		$(4 + 1) \bmod 11 = 5$	5	[22, , , , 4, 15, , , 31, 10]
28	6	$(6 + 0) \bmod 11 = 6$	6	[22, , , , 4, 15, 28, , , 31, 10]
17	6	$(6 + 0) \bmod 11 = 6$ (collision with 28)		
		$(6 + 1) \bmod 11 = 7$	7	[22, , , , 4, 15, 28, 17, , 31, 10]
88	0	$(0 + 0) \bmod 11 = 0$ (collision with 22)		
		$(0 + 1) \bmod 11 = 1$	1	[22, 88, , , 4, 15, 28, 17, , 31, 10]
59	4	$(4 + 0) \bmod 11 = 4$ (collision with 4)		
		$(4 + 1) \bmod 11 = 5$ (collision with 15)		
		$(4 + 2) \bmod 11 = 6$ (collision with 28)		
		$(4 + 3) \bmod 11 = 7$ (collision with 17)		

		$(4 + 4) \bmod 11 = 8$	8	[22, 88, , , 4, 15, 28, 17, 59, 31, 10]
--	--	------------------------	---	---

Final Hash Table (Linear Probing): [22, 88, None, None, 4, 15, 28, 17, 59, 31, 10]

1.2. Double Hashing

Double hashing uses a second hash function, $h_2(k)$, to determine the step size for probing. When a collision occurs at $h_1(k)$, the probe sequence is $(h_1(k) + i * h_2(k)) \bmod m$. This method helps to reduce primary clustering observed in linear probing.

Given $m = 11$, $h_1(k) = k \bmod 11$, and $h_2(k) = 1 + (k \bmod (m-1))$ which is $1 + (k \bmod 10)$:

Key	$h_1(k)$	$h_2(k)$	Probe Sequence (pos = $(h_1(k) + i * h_2(k)) \bmod 11$)	Final Index	Hash Table State (after insertion)
10	10	1	$(10 + 0 * 1) \bmod 11 = 10$	10	[, , , , , , , , , 10]
22	0	3	$(0 + 0 * 3) \bmod 11 = 0$	0	[22, , , , , , , , , 10]
31	9	2	$(9 + 0 * 2) \bmod 11 = 9$	9	[22, , , , , , , 31, 10]
4	4	5	$(4 + 0 * 5) \bmod 11 = 4$	4	[22, , , , 4, , , , 31, 10]
15	4	6	$(4 + 0 * 6) \bmod 11 = 4$ (collision with 4)		
			$(4 + 1 * 6) \bmod 11 = 10$ (collision with 10)		
			$(4 + 2 * 6) \bmod 11 = 16 \bmod 11 = 5$	5	[22, , , , 4, 15, , , , 31, 10]
28	6	9	$(6 + 0 * 9) \bmod 11 = 6$	6	[22, , , , 4, 15, 28, , , 31, 10]
17	6	8	$(6 + 0 * 8) \bmod 11 = 6$ (collision with 28)		
			$(6 + 1 * 8) \bmod 11 = 14 \bmod 11 = 3$	3	[22, , , 17, 4, 15, 28, , , 31, 10]

88	0	9	$(0 + 0 * 9) \bmod 11 = 0$ (collision with 22)		
			$(0 + 1 * 9) \bmod 11 = 9$ (collision with 31)		
			$(0 + 2 * 9) \bmod 11 = 18 \bmod 11 = 7$	7	[22, , 17, 4, 15, 28, 88, , 31, 10]
59	4	10	$(4 + 0 * 10) \bmod 11 = 4$ (collision with 4)		
			$(4 + 1 * 10) \bmod 11 = 14 \bmod 11 = 3$ (collision with 17)		
			$(4 + 2 * 10) \bmod 11 = 24 \bmod 11 = 2$	2	[22, , 59, 17, 4, 15, 28, 88, , 31, 10]

Final Hash Table (Double Hashing): [22, None, 59, 17, 4, 15, 28, 88, None, 31, 10]

1.3. Comparison

Comparing the final hash tables, both methods successfully inserted all keys. However, the distribution of keys and the number of probes differed. Linear probing tends to create clusters of occupied slots, leading to longer probe sequences for subsequent insertions. Double hashing, by using a second hash function for the step size, generally provides a more uniform distribution of keys and reduces clustering, often resulting in better performance in terms of average probe length.

2. Task 2: Hash Table Implementation and Search

Problem Statement: Write a program to insert a sequence of items into a hash table using the hash function $h_1(k) = 2k \bmod m$, where k is the key and m is the size of the hash table. Use linear probing for collision resolution. Then, implement a search function to find an item, showing its index if found, or an error message otherwise.

2.1. Java Implementation

The following Java code defines a `HashTable` class that implements the specified hash function and linear probing for insertion and search operations.

```
public class HashTable {
    private Integer[] table;
    private int m;

    public HashTable(int m) {
        this.m = m;
        this.table = new Integer[m];
    }

    private int hashFunc(int k) {
        //  $h_1(k) = 2k \bmod m$ 
        return (2 * k) % m;
    }

    public int insert(int k) {
        int h = hashFunc(k);
        int i = 0;
        while (i < m) {
            int pos = (h + i) % m;
            if (table[pos] == null) {
                table[pos] = k;
                return pos;
            }
            i++;
        }
        return -1; // Table is full
    }

    public int find(int k) {
        int h = hashFunc(k);
        int i = 0;
        while (i < m) {
            int pos = (h + i) % m;
            if (table[pos] != null && table[pos] == k) {
                return pos;
            }
        }
    }
}
```

```

        // If we encounter an empty slot, the item is not in the table if (table[pos]
        == null) {
            break;
        }
        i++;
    }
    return -1; // Item not found
}
public void printTable() {
    System.out.print("Final Table: [");
    for (int i = 0; i < m; i++) {
        if (table[i] == null) {
            System.out.print("None");
        } else {
            System.out.print(table[i]);
        }
        if (i < m - 1) {
            System.out.print(", ");
        }
    }
    System.out.println("]");
}

public static void main(String[] args) {
    // Task 2 Example Test
    int m2 = 9;
    int[] keys2 = {45, 23, 102, 96, 116, 87};
    HashTable ht = new HashTable(m2);
    System.out.println("\n--- Task 2: Program Simulation (Java) ---");
    for (int k :
keys2) {
        int idx = ht.insert(k);
        System.out.println("Inserted " + k + " at " + idx);
    }
    ht.printTable();

    int target1 = 87;
    int idx1 = ht.find(target1);
    System.out.println("Find " + target1 + ": " + (idx1 != -1 ? "index " + idx1 : "item not
found"));

    int target2 = 100;
    int idx2 = ht.find(target2);

```

```

        System.out.println("Find " + target2 + ": " + (idx2 != -1 ? "index " + idx2 : "item not
found"));
    }
}

```

2.2. Example Simulation

Using the example provided: $m = 9$, keys $k = \{45, 23, 102, 96, 116, 87\}$. **Insertion**

Process:

Key	$h1(k) = 2k \bmod 9$	Probe Sequence ($pos = (h1(k) + i) \bmod 9$)	Final Index
45	$(2 * 45) \bmod 9 = 90 \bmod 9 = 0$	$(0 + 0) \bmod 9 = 0$	0
23	$(2 * 23) \bmod 9 = 46 \bmod 9 = 1$	$(1 + 0) \bmod 9 = 1$	1
102	$(2 * 102) \bmod 9 = 204 \bmod 9 = 6$	$(6 + 0) \bmod 9 = 6$	6
96	$(2 * 96) \bmod 9 = 192 \bmod 9 = 3$	$(3 + 0) \bmod 9 = 3$	3
116	$(2 * 116) \bmod 9 = 232 \bmod 9 = 7$	$(7 + 0) \bmod 9 = 7$	7
87	$(2 * 87) \bmod 9 = 174 \bmod 9 = 3$	$(3 + 0) \bmod 9 = 3$ (collision with 96)	
		$(3 + 1) \bmod 9 = 4$	4

Final Hash Table (after insertions): [45, 23, None, 96, 87, None, 102, 116, None]

Search Operations:

Find 87:

$$h1(87) = (2 * 87) \bmod 9 = 174 \bmod 9 = 3$$

Probe $(3 + 0) \bmod 9 = 3$ (contains 96, not 87)

Probe $(3 + 1) \bmod 9 = 4$ (contains 87)

Output: index 4

Find 100:

$$h1(100) = (2 * 100) \bmod 9 = 200 \bmod 9 = 2$$

Probe $(2 + 0) \bmod 9 = 2$ (contains None)

Output: item not found

These results match the example output provided in the assignment, confirming the correctness of the implementation and simulation.

3. Conclusion

This assignment provided a practical understanding of hashing techniques, specifically open addressing with linear probing and double hashing. Task 1 demonstrated how different collision resolution strategies lead to varying key distributions within a hash table. Linear probing, while simple, can suffer from primary clustering, whereas double hashing offers a more dispersed probing sequence. Task 2

involved implementing a hash table with linear probing in Java, successfully demonstrating insertion and search operations with collision handling. The simulations and code implementation highlight the importance of efficient collision resolution for maintaining the performance of hash-based data structures.